

Backend Development - Learning Resources

Duration: 1.5 Months (6 Weeks)

Stack: Node.js, Express, PostgreSQL, Prisma, Redis, Docker

Commitment: 2 hours/day (achievable timeline)

WEEK 1 - Backend & Web Foundations

HTTP, REST & Web Fundamentals

Time: ~6 hours total

1. **MDN Web Docs - HTTP Overview** (Read: 1-2 hours)

- Link: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
- What to learn: HTTP methods, headers, status codes, request/response cycle
- Key sections: HTTP Messages, Methods, Status Codes

2. **freeCodeCamp - REST API Crash Course** (Video: 2-3 hours)

- Link: <https://www.youtube.com/watch?v=qbLc5a9jdXo>
- What to learn: REST principles, API design, HTTP verbs
- Hands-on: Build small REST endpoints

3. **RESTful API Best Practices** (Read: 30 mins)

- Link: <https://restfulapi.net/>
- Focus on: Resource naming, status codes, pagination
- Use as reference

4. **Postman Learning Center** (Interactive: 1 hour)

- Link: <https://learning.postman.com/docs/getting-started/introduction/>
- What to do: Create collection, test API requests, inspect responses
- Practice: Test public APIs (JSONPlaceholder, OpenWeather)

(Also look into the REST implementation using async JS)

Node.js Runtime & Event Loop

Time: ~4 hours total

1. **Official Node.js Documentation - Event Loop** (Read: 1 hour)

- Link: <https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick>
- What to learn: Event loop phases, microtasks vs macrotasks, non-blocking I/O
- Key sections: Phases, process.nextTick, setImmediate

2. **freeCodeCamp - Node.js Event Loop Deep Dive** (Article: 45 mins)

- Link: <https://www.freecodecamp.org/news/how-the-nodejs-event-loop-works/>
- What to learn: Conceptual understanding with analogies
- Practice: Write async code and predict execution order

3. **YouTube - Event Loop Masterclass** (Video: 30 mins)

- Link: <https://www.youtube.com/watch?v=T1NQWLVC5c>
- What to learn: Visual explanation of event loop phases
- Follow along with examples

4. **Node.js Official Guides** (Read: 1.5 hours)

- Link: <https://nodejs.org/en/docs/guides/>
- Read: Blocking vs Non-blocking I/O, npm packages
- Focus: Understanding module system, npm basics

WEEK 2 - Express Backend Architecture

Express Fundamentals & Routing

Time: ~5 hours total

1. **Express Official Documentation** (Read: 2-3 hours)

- Link: <https://expressjs.com/>
- What to learn: Routing, middleware, request/response objects
- Key sections: Getting Started, Basic Routing, Using Middleware

2. **freeCodeCamp - Express.js Course** (Video: 1-2 hours)

- Link: <https://www.freecodecamp.org/learn/back-end-development-and-apis/>
- What to learn: Setting up Express, routing patterns, middleware
- Complete: Basic routing exercises

3. **Express Middleware Guide** (Read: 45 mins)

- Link: <https://expressjs.com/en/guide/using-middleware.html>
- What to learn: Middleware execution order, error handling middleware
- Practice: Create custom middleware (logging, authentication)

Request Validation with Zod

Time: ~3 hours total

1. Official Zod Documentation (Read: 1 hour)

- Link: <https://zod.dev>
- What to learn: Basic schemas, validation methods, error handling
- Key sections: Introduction, Basic types, Async validation

2. Zod Schema Validation Tutorial ([Watch](#) + Code: 1.5 hours)

- Covers: Creating schemas, `.parse()` vs `.safeParse()`, nested objects
- Practice: Write schemas for User, Post, Comment models
- Source: Community tutorials and examples

3. Zod with Express (Code + Practice: 45 mins)

- Integrate Zod into Express routes
- Create validation middleware for request body, query, params
- Test with Postman

Layered Architecture

Time: ~2 hours total

1. Backend Architecture Best Practices (Read: 1 hour)

- Link: <https://refactoring.guru/design-patterns>
- Search: "Layered Architecture"
- What to learn: Controllers, Services, Repositories, Data Access Layer
- Concept: Separation of concerns

2. Practical Implementation (Code: 1 hour)

- Apply pattern to User, Post, Comment resources
 - Structure: `/controllers`, `/services`, `/repositories`
 - Practice: Move all business logic to services
-

WEEK 3 - Databases (PostgreSQL & Prisma)

PostgreSQL Fundamentals

Time: ~6 hours total

1. PostgreSQL Official Documentation (Read: 2 hours)

- Link: <https://www.postgresql.org/docs/>
- What to learn: SELECT, INSERT, UPDATE, DELETE, JOINS
- Practice: Write queries in pgAdmin or psql CLI

2. PostgreSQL Tutorial Website (Interactive: 2 hours)

- Link: <https://www.postgresqltutorial.com/>
- What to learn: Database design, normalization, constraints, transactions
- Sections: Getting Started, SQL Basics, Joins

3. SQL Joins Visual Guide (Read: 1 hour)

- Link: <https://www.postgresqltutorial.com/postgresql-tutorial/postgresql-joins/>
- What to learn: INNER, LEFT, RIGHT, FULL joins
- Practice: Write join queries for User-Posts relationships

4. Indexes & Query Performance (Read: 1 hour)

- Link: <https://www.postgresqltutorial.com/postgresql-indexes/>
- What to learn: B-Tree indexes, EXPLAIN, query optimization basics
- Practice: Analyze slow queries with EXPLAIN

Prisma ORM Mastery

Time: ~5 hours total

1. Official Prisma Documentation (Read: 2 hours)

- Link: <https://www.prisma.io/docs/orm/overview/introduction/what-is-prisma>
- What to learn: Models, migrations, Prisma Client queries
- Key sections: Getting Started, Concepts, Schema

2. Prisma Setup Tutorial (Code: 1.5 hours)

- Steps:
 - Install Prisma & PostgreSQL driver
 - Create `.env` with DATABASE_URL
 - Initialize schema with `npx prisma init`

- Write User, Post models

3. Prisma Relationships (Code: 1.5 hours)

- Link: <https://www.prisma.io/docs/orm/prisma-schema/relations>
 - What to learn: One-to-Many, Many-to-Many, One-to-One
 - Practice: Define User (many Posts), User (many Comments), Post (many Comments)
 - Practice: Query with `.include()` and `.select()`
-

Database Migrations

Time: ~2 hours total

1. Prisma Migrations Guide (Read: 45 mins)

- Link: <https://www.prisma.io/docs/orm/prisma-migrate/understanding-prisma-migrate>
- What to learn: Creating, applying, rolling back migrations
- Key: `migrate dev`, `migrate deploy`, `migrate resolve`

2. Hands-on Migration Practice (Code: 1.5 hours)

- Create initial schema
 - Run `npx prisma migrate dev --name init`
 - Modify schema (add field to User)
 - Create new migration
 - Apply migrations
 - Understand migration files
-

WEEK 4 - Authentication & Security

JWT Authentication

Time: ~4 hours total

1. JWT Introduction & Best Practices (Read: 1 hour)

- Link: <https://jwt.io/introduction>
- What to learn: JWT structure (header, payload, signature), use cases
- RFC: <https://tools.ietf.org/html/rfc7519>

2. JWT Authentication Tutorial (Code: 2 hours)

- What to implement:
 - User registration with password hashing
 - Login endpoint that returns JWT
 - Protected routes using JWT verification
 - Token expiration handling

3. **jsonwebtoken Library** (Read + Code: 1 hour)

- Link: <https://github.com/auth0/node-jwebtoken>
 - What to learn: `.sign()`, `.verify()`, expiration, refresh tokens
 - Practice: Implement token refresh logic
-

Password Security with bcrypt

Time: ~2 hours total

1. **bcrypt Best Practices** (Read: 30 mins)

- Link: <https://github.com/kelektiv/node.bcrypt.js>
- What to learn: Hashing, salting, cost factors
- Understanding: Why bcrypt over plain encryption

2. **bcrypt Implementation** (Code: 1.5 hours)

- What to do:
 - Hash passwords on signup
 - Compare passwords on login
 - Never store plain passwords
 - Practice: Test with known hashes
-

Authorization & RBAC

Time: ~3 hours total

1. **Role-Based Access Control** (Read: 1 hour)

- Link: https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html
- What to learn: Roles, permissions, authorization patterns
- Concept: Admin, member, viewer roles

2. **Express Authorization Middleware** (Code: 2 hours)

- Create role-checking middleware
 - Implement in routes (e.g., admin-only endpoints)
 - Practice: Create User, Post with owner-based permissions
 - Test: Try accessing restricted endpoints
-

Security Best Practices

Time: ~3 hours total

1. Helmet Security Headers (Code: 1 hour)

- What to do:
 - Install helmet: `npm install helmet`
 - Add `app.use(helmet())`
 - Understand security headers being set

2. CORS Configuration (Code: 1 hour)

- What to do:
 - Install cors: `npm install cors`
 - Configure allowed origins
 - Set credentials and methods
 - Test with curl and browser

3. OWASP Top 10 Overview (Read: 1 hour)

- Link: <https://owasp.org/www-project-top-ten/>
 - What to learn: SQL injection, XSS, CSRF, input validation
 - Focus: Why input validation matters, preventing common attacks
-

WEEK 5 - Performance & Asynchronous Systems

Redis Caching

Time: ~4 hours total

1. Redis Official Documentation (Read: 1.5 hours)

- Link: <https://redis.io/documentation>
- What to learn: Data types (strings, lists, hashes, sets), TTL, expiration
- Key sections: Overview, Data Types, Commands

2. Redis Setup & Basics (Code along + [Tutorial](#): 1.5 hours)

- Install: `npm install redis`
- What to do:
 - Connect to local Redis (or Docker)
 - SET/GET operations
 - EXPIRE commands
 - Practice basic operations in Node.js

3. Caching Patterns in Express (Code: 1 hour)

- Implement cache-aside pattern
 - Cache API responses (e.g., GET /users/:id)
 - Invalidate cache on updates (POST, PATCH, DELETE)
 - Practice: Add caching to 3-4 endpoints
-

Background Jobs with BullMQ

Time: ~4 hours total

1. BullMQ Official Documentation (Read: 1 hour)

- Link: <https://docs.bullmq.io/>
- What to learn: Queues, jobs, workers, events
- Key sections: Introduction, Concepts, Quick Start

2. BullMQ Setup & Implementation (Code: 2 hours)

- What to implement:
 - Create queue for async tasks
 - Add job to queue (e.g., send email)
 - Create worker to process jobs
 - Handle job completion and failures

3. Job Queues in Express API (Code: 1 hour)

- Trigger async job from API endpoint
 - Track job status
 - Practice: Implement email sending, image processing jobs
-

Advanced Async Patterns

Time: ~2 hours total

1. Promises & Async/Await Deep Dive (Read: 45 mins)

- Link: <https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous>
- What to learn: Error handling with try/catch, Promise.all variations

2. **Real-World Async Patterns** (Code: 1.5 hours)

- Handle concurrent operations efficiently
 - Implement retry logic with exponential backoff
 - Practice: Fetch from multiple APIs simultaneously
-

WEEK 6 - Production Backend Engineering

Docker Containerization

Time: ~4 hours total

1. **Docker Official Documentation** (Read: 1.5 hours)

- Link: <https://docs.docker.com/get-started/>
- What to learn: Images, containers, layers, Dockerfile syntax
- Key sections: Getting Started, Concepts

2. **Docker Tutorial for Node.js** (Code: 2 hours)

- What to do:
 - Create Dockerfile for Express app
 - Build image: `docker build -t myapp .`
 - Run container: `docker run -p 3000:3000 myapp`
 - Optimize Dockerfile (multi-stage builds)

3. **Docker Best Practices** (Read: 30 mins)

- Link: <https://docs.docker.com/develop/dev-best-practices/>
 - What to learn: Minimizing image size, security
-

Docker Compose Multi-Service Setup

Time: ~3 hours total

1. **Docker Compose Documentation** (Read: 1 hour)

- Link: <https://docs.docker.com/compose/>
- What to learn: Services, networking, volumes, environment variables
- Key sections: Getting Started, Compose File Reference

2. Docker Compose with PostgreSQL + Redis (Code: 2 hours)

- What to setup:
 - PostgreSQL service
 - Redis service
 - Node.js Express service
 - Volume for database persistence
 - Run: `docker-compose up`
-

Structured Logging

Time: ~3 hours total

1. Winston Logger (Read + Code: 1.5 hours)

- What to implement:
 - Set up Winston with JSON format
 - Log at different levels (info, warn, error)
 - Add context fields (userId, requestId)
 - Use in all endpoints

2. Pino Logger (Read + Code: 1 hour)

- Compare with Winston
- Optional: Integrate Pino if preferred

3. Structured Logging Best Practices (Read: 30 mins)

- Link: <https://www.kartar.net/2015/12/structured-logging/>
 - What to learn: What to log, avoiding sensitive data
-

Monitoring & Observability (Conceptual)

Time: ~2 hours total

1. Application Monitoring Basics (Read: 1 hour)

- Link: <https://newrelic.com/blog/best-practices/application-monitoring>
- What to learn: Metrics, logs, traces (three pillars of observability)
- Concept understanding (no implementation required)

2. Health Checks & Graceful Shutdown (Code: 1 hour)

- Implement `/health` endpoint

- Return db connectivity status
 - Graceful shutdown on SIGTERM
 - Practice: Test container stopping
-

Deployment Fundamentals

Time: ~2 hours total

1. Deployment Platforms Overview (Read: 45 mins)

- Platforms: Railway.app, Render, Vercel, AWS
- Compare features, pricing, ease of use
- Choose one to deploy on

2. Deploy to Railway or Render (Code: 1.5 hours)

- Follow platform-specific guide
 - Deploy your Docker app
 - Set environment variables
 - Test live API
 - Practice: Deploy the capstone project
-

System Design Fundamentals (Conceptual)

Time: ~3 hours total

1. CAP Theorem & Distributed Systems (Read: 1 hour)

- Link: https://en.wikipedia.org/wiki/CAP_theorem
- Simpler read: <https://www.educative.io/blog/cap-theorem>
- What to learn: Consistency, Availability, Partition tolerance

2. Scaling Concepts (Read: 1.5 hours)

- Link: <https://www.educative.io/blog/web-application-system-design>
- Topics: Load balancing, read replicas, caching layers, database sharding
- Understand concepts (no implementation)

3. System Design Interview Questions (video: 30 mins)

- Link: <https://www.youtube.com/watch?v=q0KGYwNbf-0> (System Design Primer)
- Watch: Design Twitter, Uber overview

- Goal: Exposure to large-scale thinking
-

THROUGHOUT ALL WEEKS - Practice & Projects

Hands-On Projects & Practice

1. Postman Practice (Daily)

- Test all your API endpoints
- Create automated test collections
- Practice with public APIs

2. GitHub Repository

- Commit code daily
- Write meaningful commit messages
- Practice: `git commit`, `git branch`, `git merge`
- Link: <https://github.com/>

3. DevCollab API Capstone (Ongoing - Weeks 1-6)

- Build incrementally each week
 - Week 1: Basic Express server with routes
 - Week 2: Add validation, layered architecture
 - Week 3: Connect to PostgreSQL, Prisma
 - Week 4: Add authentication, authorization
 - Week 5: Add caching, background jobs
 - Week 6: Containerize, deploy
-

ADDITIONAL REFERENCE RESOURCES

Official Documentation (Always Available)

- Node.js: <https://nodejs.org/docs>
- Express: <https://expressjs.com>
- PostgreSQL: <https://www.postgresql.org/docs/>
- Prisma: <https://www.prisma.io/docs>
- Redis: <https://redis.io/documentation>
- Docker: <https://docs.docker.com>

Community & Help

- Stack Overflow: <https://stackoverflow.com/questions/tagged/node.js>

- GitHub Discussions: <https://github.com/> (in repo projects)
- Postman Community: <https://www.postman.com/community/>

Advanced Topics (After Week 6)

- TypeScript with Node.js: <https://www.typescriptlang.org/>
 - NestJS Framework: <https://docs.nestjs.com/>
 - GraphQL: <https://graphql.org/learn/>
 - Testing: <https://jestjs.io/docs/getting-started>
-

SUCCESS METRICS

By end of Week 6, you should:

-  Have built DevCollab API (or similar full-featured backend)
 -  Deploy working backend to production
 -  Use PostgreSQL, Redis, Docker
 -  Implement authentication, authorization, RBAC
 -  Write structured logs
 -  Understand backend architecture patterns
 -  Ready for backend engineering interviews
-

QUICK LINKS BY TOPIC

HTTP & REST:

- MDN HTTP: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
- REST API Course: <https://www.youtube.com/watch?v=qbLc5a9jdXo>

Node.js:

- Event Loop: <https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick>
- Official Docs: <https://nodejs.org/docs>

Express:

- Official Docs: <https://expressjs.com>
- freeCodeCamp Course: <https://www.freecodecamp.org/learn/back-end-development-and-apis/>

PostgreSQL & Prisma:

- PostgreSQL Tutorial: <https://www.postgresqltutorial.com/>
- Prisma Docs: <https://www.prisma.io/docs>

Redis:

- Redis Docs: <https://redis.io/documentation>
- BullMQ Docs: <https://docs.bullmq.io/>

Docker:

- Docker Docs: <https://docs.docker.com/get-started/>
- Docker Compose: <https://docs.docker.com/compose/>

Security:

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- Helmet Docs: <https://helmetjs.github.io/>

Good luck! 🚀